

FAST – NUCES



Object Oriented Programming Lab Manual

Course Teacher	Ms. Hina Iqbal
Lab Instructor	Mr. Ahmad Jawad Mustasim
Lab TA	Ms. Mania Shakeel
Section	BSE-2B
Semester	Spring 2026

Department of Computer Science

FAST-NU, Lahore, Pakistan

Lab 2 – Pointers/ functions

Objectives

- Understanding and implementing pointers
 - Use of const in Pointers
 - 1D dynamic arrays
 - Pointers in functions
-



Read carefully and attempt following tasks

1. Plankton's Energy Analyzer (Pointer to Const Array)

Plankton is tracking the average energy consumption of Bikini Bottom machines. He wants a function that calculates the average safely without changing original data

Function Prototype

```
float average(const float* arr, int size);
```

Requirements

- Use **pointer to const float**
- Array values **must NOT be modified** inside the function
- In main():
 - Ask user for number of elements
 - Create a **dynamic float array**
 - Take energy values from user
 - Pass dynamic array to function
- Function should calculate and return average energy consumption

Story Twist

Plankton is paranoid — he does NOT trust anyone to modify his data. So your function must promise: **read-only access only!**

2. Squidward's Mood Sorter (Bubble Sort)

Squidward wants to arrange his clarinet moods in ascending order. Each mood has a **power level** (float).

Write a C++ program that:

- Asks the user for number of moods (elements)
- Reads all mood power levels into a **dynamic array**
- Write a function to sort them using **bubble sort**
- Display sorted power levels after sorting

Squidward demands:

Everything must be in **ascending order** or he refuses to perform.

3. SpongeBob's Krabby Message Reorder (Pointer String Manipulation)

SpongeBob received a secret Krabby Patty mission message.

He wants to rearrange it using **only pointers** — no extra memory allowed!

Function to Write

```
void concat(const char* src, char* dest);
```

Strict Rules

- src → pointer to const char (**read-only**)
- dest → writable character array
- **NO extra arrays or buffers**
- **NO built-in string functions** (no strcpy, strlen, etc.)
- **NO destSize parameter**
- Calculate string lengths manually (you may create helper function)

Steps

Assume:

- dest already contains a string
- src contains another string

Your function must:

1. Shift existing characters of dest forward to make space
2. Copy all characters of src at the **beginning**
3. Insert one space ' ' after src
4. Append original contents of dest after space
5. Properly place null character '\0'
6. Display updated dest **inside the function**

Final Format Required

```
[src][space][old dest]
```

Example (Hardcoded in main)

```
char src[] = "SpongeBob";
```

```
char dest[] = "SquarePants";
```

After calling:

```
concat(src, dest);
```

Output (Printed inside function)

```
SpongeBob SquarePants
```

Bikini Bottom Rule

SpongeBob cannot afford extra memory in his pineapple house —

so everything must be done using **only pointers and existing space!**

4. Patrick's Cumulative Snack Power

Patrick wants to track his **total snack power** during the day.

Each snack has a power level (float).

Write a C++ program that:

- Ask the user for number of snacks Patrick ate
- Store snack powers in a **dynamic array**
- Create another dynamic array where each index stores **total power from snack 1 up to that index** (cumulative sum)
- Display the cumulative power array

Patrick loves knowing how powerful he becomes after each snack.

5. Mr. Krabs' Employee ID Cleaner (Remove Duplicates)

Mr. Krabs keeps employee IDs on a list.

Sometimes he accidentally writes the same employee twice — and he hates paying twice!

Write a C++ program that:

- Ask the user for size of employee list
- Input employee IDs (integers) into a **dynamic array**
- Remove all duplicate IDs
- Display final employee IDs after duplicates are removed

Mr. Krabs Rule:

No duplicate salaries allowed!

6. SpongeBob's Expanding Jellyfish Net (Dynamic Resizing)

SpongeBob is catching jellyfish

His net has a fixed capacity, but sometimes he catches more than it can hold.

Write a C++ program that:

- Ask the user for initial size of jellyfish net
- Keep taking jellyfish IDs (integers) from user
- Stop when user enters -1
- If number of jellyfish exceeds current size:
 - **Double the size** of the net
 - Continue storing remaining jellyfish
- Display all jellyfish IDs in the net

SpongeBob Rule:

When the net fills → **expand it and keep catching!**

-----Happy Coding 😊-----



Submission Instructions

- Submit your tasks on Google Classroom within given deadline
- Submit solution file of each task separately for example “LAB01-TASK01.cpp”, “LAB01-TASK02.cpp” and so on
- Also submit screenshots of each task for example “LAB01-TASK01.png”, “LAB01-TASK02.png” and so on
- “.cpp” is the actual file where all your code is saved.
- Always submit .cpp files when asked for submission
- Example path of .cpp file is

Desktop\Your_Roll_No\LAB01-TASK01\LAB01-TASK01\LAB01-TASK01.cpp

Copy your files from this path



